Spring Boot Codebase Guide

Purpose: internal codebase explanation for studying this repository as a real Spring Boot backend.

Focus: folder structure, file responsibilities, runtime flow, security flow, configuration, persistence, and test layers.

This guide reflects the final scope of Spring Boot Learning Phase 01 and intentionally stays focused on the user-management backend.

1. Project Purpose

This repository is a learning-to-production-style Spring Boot backend. It starts with simple REST endpoints and grows into a more complete API with DTOs, validation, exception handling, JWT security, role-based authorization, pagination, search, logging, Swagger, profiles, Flyway, and multiple testing layers.

Main business area

User management with authentication.

- `POST /auth/login`
- `GET /` and `GET /hello`
- `POST /users`
- `GET /users`
- `GET /users/{id}`
- `PUT /users/{id}`
- `DELETE /users/{id}`
- `GET /users/search`

Main technical themes

- Layered architecture
- DTO-driven API design
- Validation and global exception handling
- JWT + RBAC security
- JPA + Flyway persistence
- Profiles and environment separation
- Unit, slice, integration, and security tests

2. Folder Structure

```
springboot-journey
|- docs
|  |- java-springboot-interview-pack.html
|  |- java-springboot-interview-pack.pdf
|  `- springboot-codebase-guide.html / pdf
|- src
|  |- main
|  |  |- java/com/isanjalee/demo/springbootdemo
|  |  |  |- config
|  |  |  |- controller
|  |  |  |- dto
|  |  |  |- exception
|  |  |  |- model
|  |  |  |- repository
|  |  |  |- security
|  |  |  |- service
|  |  |  `- SpringbootdemoApplication.java
|  |  `- resources
|  |     |- application.properties
|  |     |- application-dev.properties
|  |     |- application-prod.properties
|  |     |- application-docker.properties
|  |     `- db/migration/V1__create_users_table.sql
|  `- test/java/com/isanjalee/demo/springbootdemo
|     |- controller
|     |- repository
|     |- service
|     |- HelloWorldTest.java
|     `- SpringbootdemoApplicationTests.java
|- pom.xml
|- Dockerfile
|- docker-compose.yml
`- README.md
```

How to read this codebase: start with `SpringbootdemoApplication`, then read configuration, then controllers, services, repository, entity, resources, and finally tests.

---

3. High-Level Architecture

> Client Request -> Spring Security Filter Chain -> `JwtAuthFilter` -> Controller -> DTO validation -> Service -> Repository -> Database -> Service maps entity to DTO -> Controller returns HTTP response

There are three important side paths in this flow:

- If validation fails, Spring throws a validation exception before service logic runs.
- If business rules fail, custom exceptions are thrown in the service layer.
- If security fails, Spring Security returns 401 or 403 through configured handlers.

---

4. Entry Point and Bootstrapping

File	Role	Important details
`SpringbootdemoApplication.java`	Main entry point	Starts Spring Boot, enables configuration property scanning, and enables Spring caching.
`pom.xml`	Build and dependency definition	Declares Spring Boot starters, JPA, Security, Validation, Flyway, H2, PostgreSQL, SpringDoc, test dependencies, and JaCoCo coverage reporting.
`README.md`	Project overview	Summarizes features, profiles, endpoints, and test commands.

---

5. Main Java Packages and Files

5.1 config

File	What it does	How it is used
`AppConfig.java`	Declares a `PasswordEncoder` bean.	`AuthService` uses it to verify passwords, and `UserService` uses it to hash new passwords.
`SecurityConfig.java`	Defines the security filter chain.	Disables CSRF, makes the API stateless, permits auth/docs/H2 routes, protects everything else, wires custom 401/403 handlers, and inserts `JwtAuthFilter` before the username-password filter.
`JwtProperties.java`	Maps `app.jwt.*` properties.	`JwtUtil` uses it to read the signing secret and expiry time.

File	What it does	How it is used
`DevCacheConfig.java`	Creates an in-memory cache manager for the `dev` profile.	`UserService` cache annotations use the `userById` and `usersPage` caches during development.
`DataSeeder.java`	Seeds a default admin user in the `dev` profile.	Runs at startup via `CommandLineRunner` and inserts `admin@gmail.com / admin123` if not present.
`RedisCacheConfig.java`	Placeholder for Redis-based production caching.	Currently commented out, so it documents a future direction rather than running in the active application.

5.2 controller

File	Endpoints	Role in the flow
`HelloController.java`	`GET /`, `GET /hello`	Simple protected endpoints used to prove the app and security are working.
`AuthController.java`	`POST /auth/login`	Accepts validated login input and delegates authentication to `AuthService`.
`UserController.java`	`/users` CRUD + search	Thin HTTP layer that accepts DTOs, uses `@PreAuthorize` for admin-only access, passes work to `UserService`, and returns proper responses.

5.3 dto, exception, model, repository, security, service

Area	Files	Main responsibility
DTOs	`LoginRequest`, `LoginResponse`, `UserCreateRequest`, `UserUpdateRequest`, `UserResponse`, `ErrorResponse`	Define request, response, and error payload shapes. Validation annotations live here for input DTOs.
Exceptions	`BadRequestException`, `DuplicateResourceException`, `ResourceNotFoundException`, `GlobalExceptionHandler`	Represent domain errors and convert them into consistent HTTP responses.
Model	`User.java`	JPA entity mapped to the `users` table.

Area	Files	Main responsibility
Repository	`UserRepository.java`	Data access through JPA, plus email lookup and search helpers.
Security	`JwtUtil`, `JwtAuthFilter`, `RestAuthenticationEntryPoint`, `RestAccessDeniedHandler`	Generate tokens, validate them on requests, and standardize 401/403 responses.
Services	`AuthService`, `UserService`	Contain login logic, user business rules, password hashing, duplicate checks, DTO mapping, cache usage, and search logic.

6. Resources and Environment Files

File	Purpose	What matters most
`application.properties`	Shared default configuration.	Defines base datasource, JWT values, Flyway, logging, actuator, and default active profile `dev`.
`application-dev.properties`	Development profile settings.	Uses H2 and simple local cache with debug-friendly logging.
`application-prod.properties`	Production-style configuration.	Uses PostgreSQL-style environment variables, disables H2 console, and uses safer runtime defaults.
`application-docker.properties`	Docker profile configuration.	Points to Docker-hosted PostgreSQL and Redis service names.
`db/migration/V1__create_users_table.sql`	Flyway schema migration.	Creates the `users` table and enforces unique email.

7. End-to-End Request Flows

7.1 Application Startup Flow

1. Spring Boot starts from `SpringbootdemoApplication`.
2. Configuration properties are loaded from `application*.properties`.
3. Spring creates beans from configuration classes.
4. Flyway validates and applies migrations.
5. JPA/Hibernate initializes entity mapping.
6. Security filter chain is built from `SecurityConfig`.
7. If profile is `dev`, cache manager and seeded admin user are created.

7.2 Login Flow

> `POST /auth/login` -> `AuthController` -> validate `LoginRequest` -> `AuthService.login()` -> `UserRepository.findByEmail()` -> password check -> `JwtUtil.generateToken()` -> `LoginResponse`

7.3 Secured User CRUD Flow

> Client sends bearer token -> `JwtAuthFilter` parses token -> Spring Security stores authentication -> `@PreAuthorize("hasRole('ADMIN')")` checks role -> controller method runs -> service performs business logic -> repository accesses DB -> service maps entity to DTO -> controller returns response

7.4 Validation and Error Flow

- Validation annotations are declared in DTO classes.
- `@Valid` triggers validation in controllers.
- `MethodArgumentNotValidException` is handled by `GlobalExceptionHandler`.
- Business exceptions like duplicate email and not found are also handled centrally.
- Security failures are split into 401 via the authentication entry point and 403 via access-denied handling.

7.5 Search, Pagination, and Cache Flow

`UserController.searchUsers()` accepts optional `name` or `email` query parameters and a `Pageable`. The service chooses the correct repository query and maps the result to `UserResponse`. `getUserById()` and `getAllUsers()` use cache annotations, while create, update, and delete evict those cache entries.

8. Testing Structure

Test file	Type	What it proves
`HelloWorldTest.java`	Basic unit placeholder	Very small sanity test showing the test pipeline is working.
`SpringbootdemoApplicationTests.java`	Context load test	Verifies that the Spring application context starts successfully.
`UserRepositoryTest.java`	`@DataJpaTest`	Validates custom repository behavior such as search queries and email lookup.
`UserServiceTest.java`	Unit test with Mockito	Validates service rules like duplicate checks, mapping, missing-user handling, update behavior, and delete behavior.
`AuthControllerWebMvcTest.java`	`@WebMvcTest`	Tests login-controller behavior with mocked service dependencies.
`UserControllerWebMvcTest.java`	`@WebMvcTest`	Tests admin security restrictions, validation, response shape, and invalid JWT handling for user endpoints.
`AuthControllerIntegrationTest.java`	`@SpringBootTest` + MockMvc	Tests the full login flow with real Spring beans, repository, and password encoding.
`HelloControllerIntegrationTest.java`	`@SpringBootTest` + MockMvc	Tests security around the sample endpoints.
`UserControllerIntegrationTest.java`	`@SpringBootTest` + MockMvc	Tests real end-to-end behavior for CRUD, search, validation, authorization, invalid token handling, and not-found scenarios.

Reading tests gives fast understanding: if you want to learn what the code is supposed to do, the integration tests are one of the best entry points because they show expected behavior from the outside.

9. How the Main Files Work Together

Concern	Files involved	Combined behavior
Login	`AuthController`, `LoginRequest`, `AuthService`, `UserRepository`, `PasswordEncoder`, `JwtUtil`, `LoginResponse`	Validates credentials and issues a signed JWT token.
Authorization	`SecurityConfig`, `JwtAuthFilter`, `RestAuthenticationEntryPoint`, `RestAccessDeniedHandler`, `@PreAuthorize`	Protects endpoints and converts security failures into JSON responses.
User creation	`UserController.create()`, `UserCreateRequest`, `UserService.createUser()`, `UserRepository`, `User`, `UserResponse`	Validates input, checks uniqueness, hashes password, saves the entity, and returns safe response data.
Error handling	`GlobalExceptionHandler`, `ErrorResponse`, custom exception classes	Standardizes validation, business, security, and fallback errors across the API.
Environment behavior	`application*.properties`, `JwtProperties`, `DataSeeder`, `DevCacheConfig`	Changes runtime behavior by profile without changing business code.

10. Recommended Learning Path Through This Repo

1. Read `README.md` to understand what the project claims to do.
2. Open `pom.xml` and identify the core dependencies.
3. Read `SpringbootdemoApplication.java` and config classes.
4. Read `SecurityConfig`, `JwtAuthFilter`, and `JwtUtil` together.
5. Read controllers to see available endpoints.
6. Read DTOs to understand request and response shapes.

7. Read services to understand business rules.
8. Read `UserRepository` and `User` for persistence structure.
9. Read resources and Flyway migration for environment and schema.
10. Read tests last to confirm expected behavior and edge cases.

---

11. Final Internal Summary

> This is a layered Spring Boot backend where controllers manage HTTP, services contain business rules, repositories handle persistence, DTOs define the API contract, security is enforced through JWT and role checks, exceptions are standardized globally, environment behavior is controlled by profiles, and the whole stack is validated through repository, service, WebMvc, integration, and security-focused tests.

---

12. End of Learning Spring Boot Phase 01

> This repository is complete for Phase 01 because it already covers the core Spring Boot backend learning path: application bootstrapping, REST APIs, layered architecture, DTOs, validation, exception handling, JPA, Flyway, JWT, RBAC, pagination, logging, Swagger, profiles, and several test layers. The project intentionally stops here to keep the codebase compact and reviewable.

---

13. Codebase Interview Questions With Answers

> **Q: Why is the service layer important in this codebase?**
>
> A: The service layer keeps business rules out of controllers. Controllers only handle HTTP concerns, while services manage decisions such as duplicate-email checks, password hashing, caching behavior, and not-found handling. This separation makes the code easier to test and maintain.

> **Q: How does authentication work from request to response?**
>
> A: The client sends email and password to `POST /auth/login`. The controller passes the request to `AuthService`, which loads the user from the repository, verifies the

password with `PasswordEncoder`, and generates a JWT through `JwtUtil`. That token is then returned in the login response and used on later requests.

Q: Where is validation enforced and how are errors returned?

A: Validation rules are declared in request DTOs, and controller methods trigger them with `@Valid`. If input is invalid, Spring raises a validation exception before service logic runs. `GlobalExceptionHandler` then converts that into a structured JSON response with the correct HTTP status.

Q: Why use DTOs instead of returning entities directly?

A: DTOs protect the API contract from internal entity changes, prevent accidental exposure of sensitive fields like passwords, and give a clean place for validation. They also make responses clearer because the API returns only what clients actually need.

Q: What changes between dev and prod behavior in this repo?

A: Development uses local-friendly settings such as H2, dev cache config, and seed data for fast testing. Production-style configuration switches to PostgreSQL-oriented settings, disables local-only tooling such as the H2 console, and uses safer defaults. Profiles allow those differences without changing core business code.

Q: Why are there multiple test layers instead of only integration tests?

A: Each test layer answers a different question. Unit tests check service rules quickly, repository tests check JPA behavior, WebMvc tests verify controller and security behavior in isolation, and integration tests verify the full request flow. Together they give faster feedback and better confidence than using only one style of test.

Prepared as an internal codebase study guide and aligned to the completed scope of Spring Boot Learning Phase 01. Best used while reading the project side by side in the IDE.